Benchmarks from the related work, written against the Must API

Specification and implementation pairs for TraceForge, for the implementer

The author, for the owner

2026-10-26

What this file is

The related work (`plan/RELATED-AND-BENCHMARKS-TODO.md`, verified section) evaluates on a small set of protocols: two-phase commit (Disel, Aneris, Trillium), a load balancer (Aneris), key-value stores (IronKV, Verdi, Grove, Chapar, Memcached in Bílý et al.), leader election (Igloo), Paxos (IronRSL, Trillium, MoDist's MPS), Raft (Verdi), a lock server (Verdi's repository). Our five families already cover two-phase commit, a key-value store, a lock, a load balancer and a leader election. This file gives six more pairs, the realistic protocols the related work verifies and we do not yet run, each as a specification and an implementation written in the style of the existing fixtures in `conformance/grid.rs`: threads spawned with `named`, typed channels from `typed_chan`, `send_msg`, `recv_msg_block` (blocking), `recv_msg` (non-blocking, `Option`), `nondet()` for a boolean choice and `(0..n).nondet()` for a choice in a range, thread-addressed `send_msg(tid, m)` and `recv_msg_block::<T>()` where a thread has no channel. The communication model is the run's (`asyn`, `p2p` or `cd`); each pair says which it needs. Every program is loop-free after unrolling its bounded parameters, has static threads, and declares its visible threads by name; the two programs of a pair have the same visible threads.

The programs are written for the lead to transcribe into fixtures, not to compile as they stand: they omit the `prog(move || { ...})` wrapper, the message enums' derive lines, and the plumbing functions that the existing A1–A4 fixtures already have (`reply_channels`, `spawn_clients`). Where a fixture needs a choice the Must API does not have, the text says so. For each pair: the source in the related work, what the two sides do, where they differ, the visible threads, the parameters and the smallest sizes, the faults to seed and the verdict expected for each, and what makes the pair worth running.

Four rules every pair follows (from the review of the first version of this file, Opus, 2026-10-26; the first version broke three of them).

1. **A non-blocking receive may always read $\perp$**, whether or not a message is pending. So an invisible thread that serves clients from a bounded `recv_msg` loop has graphs in which it serves nothing, and the clients it serves block. Either the specification can stop at any point too (an `if nondet() { return; }` in every iteration of its server), or the implementation's servers use blocking receives in the idiom of A4 (a `loop` that ends blocked when the clients are done, the invisible status being unobserved). The pairs below use blocking servers wherever a client's progress depends on them.

2. **One specification thread orders everything it serves.** Two operations of different clients served by one invisible thread are causally ordered in every specification graph, through that thread's program order. An implementation that serves them through independent threads has graphs with no such order, and those are reported (`prop:refstrict`). So a specification has one server thread per unit the implementation may serve independently: one per key for a sharded store, one per replica for a replicated store, one oracle for a decision every client observes.

3. **Visible threads send and receive only what they send and receive on the other side.** A routing update, a retry or a control message received by a visible client is a visible event with no counterpart; it goes to an invisible proxy.

4. **A fault must be reachable by the script.** Each fault names the schedule and the operations that expose it at the smallest size, and a fault that the script cannot expose is a conforming control, said so.

`typed_chan` is a FIFO channel and `apps_config` runs `ConsType::FIFO`; a pair that needs reordering (B1's acceptors, B4's updates) builds its channels with a `NoOrder` builder, which the lead adds to the fixture plumbing.

Contents

1 B1 — Single-decree Paxos against a consensus oracle **2**

2 B2 — Raft log replication, bounded, against an atomic log **4**

3 B3 — Sharded key-value store with key migration against per-key stores **7**

4 B4 — Causally consistent replicated key-value store against per-replica causal servers **9**

5 B5 — A Memcached-style server with connections against one map **10**

6 B6 — Ring leader election against an oracle naming the maximum **12**

7 Summary and order of implementation **13**

1 B1 — Single-decree Paxos against a consensus oracle

Source. IronFleet's IronRSL (SOSP 2015), Trillium (POPL 2024, single-decree Paxos refining Lamport's TLA$^+$ model), MoDist's MPS (NSDI 2009), Bílý et al. (OOPSLA 2025, §5.3). Design reviewed by Opus and Codex (`plan/RELATED-OOPSLA25.md` §4).

What the pair checks. The implementation runs one instance of single-decree Paxos: two proposers with ballots 1 and 2, three acceptors, each proposer carrying the value of one client. The specification is a consensus oracle: it may decide any proposed value, or nothing, and tells every learner the same decision. The visible threads are the clients, which send their proposals, and the learners, which receive the decision. Proposers and acceptors are invisible. Agreement is what the specification imposes: every learner that receives a decision receives the same value, and only a proposed one.

Messages.

```
enum ToProposer { Propose(i32) }                        // client -> proposer
enum ToAcceptor { Prepare(u8), Accept(u8, i32) }        // proposer -> acceptor, ballot first
enum ToProposerFromAcc { Promise(u8, Option<(u8, i32)>), Accepted(u8) }
enum ToLearner  { Decide(i32) }
```

Specification: the oracle.

```
// visible: c0, c1, l0, l1       invisible: oracle
let (tx_o, rx_o) = typed_chan::<ToProposer>();          // both clients send here
let (tx_l, rx_l) = learner_channels(2);                 // one channel per learner
named("oracle", move || {
    let mut vals: Vec<i32> = Vec::new();
    for _ in 0..2 {                                     // at most one proposal per client
```

```
        if let Some(ToProposer::Propose(v)) = rx_o.recv_msg() { vals.push(v); }
    }
    if !vals.is_empty() && crate::nondet() {               // decide, or decide nothing
        let v = vals[(0..vals.len()).nondet()];
        for tx in &tx_l { tx.send_msg(ToLearner::Decide(v)); }
    }
});
for i in 0..2 { let tx = tx_o.clone();
    named(&format!("c{i}"), move || tx.send_msg(ToProposer::Propose(10 + i as i32))); }
for (i, rx) in rx_l.into_iter().enumerate() {
    named(&format!("l{i}"), move || { let _d: ToLearner = rx.recv_msg_block(); }); }
```

The oracle receives each proposal non-blocking and may leave it unread. That is deliberate: every proposal the oracle consumes becomes a causal predecessor of the decision, and the implementation's decision depends causally only on the proposals that reached the winning proposer, so an oracle that consumed every proposal would impose an order the implementation does not have and produce false alarms (`prop:refstrict`). With non-blocking receives the oracle has a graph for every subset. A learner blocks when nothing is decided, on both sides.

Implementation: Paxos.

```
// visible: c0, c1, l0, l1       invisible: p1, p2, a0, a1, a2
let (tx_p, rx_p) = proposer_channels(2);     // client i -> proposer i+1 (ballot i+1)
let (tx_a, rx_a) = acceptor_channels(3);
let (tx_pa, rx_pa) = proposer_reply_channels(2);   // acceptor -> proposer, by ballot
let (tx_l, rx_l) = learner_channels(2);
for b in 1..=2u8 {
    let rx_prop = rx_p.remove(0); let rx_rep = rx_pa.remove(0);   // one receiver per proposer
    let tx_a = tx_a.clone(); let tx_l = tx_l.clone();
    named(&format!("p{b}"), move || {
        let ToProposer::Propose(mine) = rx_prop.recv_msg_block();
        for tx in &tx_a { tx.send_msg(ToAcceptor::Prepare(b)); }
        let mut promises = 0; let mut best: Option<(u8, i32)> = None;
        for _ in 0..3 {                                  // collect what arrives, bounded
            if let Some(ToProposerFromAcc::Promise(bb, acc)) = rx_rep.recv_msg() {
                if bb == b { promises += 1;
                    if let Some((ab, av)) = acc {
                        if best.map_or(true, |(x, _)| ab > x) { best = Some((ab, av)); } } } } }
        if promises < 2 { return; }                      // no quorum: give up
        let v = best.map_or(mine, |(_, av)| av);         // the Paxos rule
        for tx in &tx_a { tx.send_msg(ToAcceptor::Accept(b, v)); }
        let mut accepted = 0;
        for _ in 0..3 {
            if let Some(ToProposerFromAcc::Accepted(bb)) = rx_rep.recv_msg() {
                if bb == b { accepted += 1; } } }
        if accepted >= 2 { for tx in &tx_l { tx.send_msg(ToLearner::Decide(v)); } }
    });
}
for a in 0..3 {
    let rx = rx_a[a]; let tx_pa = tx_pa.clone();
    named(&format!("a{a}"), move || {
        let mut promised: u8 = 0; let mut accepted: Option<(u8, i32)> = None;
        for _ in 0..4 {                                  // at most 2 prepares + 2 accepts
            match rx.recv_msg() {
                Some(ToAcceptor::Prepare(b)) if b > promised => {
                    promised = b; tx_pa[b as usize - 1].send_msg(ToProposerFromAcc::Promise(b,
    accepted)); }
                Some(ToAcceptor::Accept(b, v)) if b >= promised => {
                    promised = b; accepted = Some((b, v));
                    tx_pa[b as usize - 1].send_msg(ToProposerFromAcc::Accepted(b)); }
                _ => {}                                  // stale ballot or nothing pending
            } }
    });
}
for i in 0..2 { let tx = tx_p[i].clone();
```

```
    named(&format!("c{i}"), move || tx.send_msg(ToProposer::Propose(10 + i as i32))); }
for (i, rx) in rx_l.into_iter().enumerate() {
    named(&format!("l{i}"), move || { let _d: ToLearner = rx.recv_msg_block(); }); }
```

Two proposers may both reach a decision; Paxos guarantees they decide the same value, so each learner receives one `Decide` and reads the first (the second stays pending, unobserved). If the learners should see exactly one message, make the proposers send to a single invisible `announcer` that forwards once; the oracle is unchanged.

Differences. The specification has no ballots, no quorums and no acceptors; its only nondeterminism is which proposal, if any, is decided. The implementation's nondeterminism is the message order among proposers and acceptors, which decides whether a proposer reaches a quorum and whether the second proposer adopts the first's value.

Parameters. Proposers 2, acceptors 3, clients 2, learners 2, ballots {1, 2}; the acceptors' loop bound 4 (two prepares and two accepts) and the proposers' collection bounds 3 (one per acceptor) are what unrolls the protocol, and a late promise eating one accept-phase iteration still leaves a quorum. Communication: the acceptor and proposer channels `NoOrder` (a Paxos network reorders), the client and learner channels FIFO. Growth: a third proposer with ballot 3, or a third client, or three learners.

Faults.

1. `ignore-best`: the proposer uses `mine` even when `best` is `Some` (drop the Paxos rule). Expected: a run in which ballot 1 decides v_1 and ballot 2 decides $v_2 \neq v_1$; two learners observe different values; uncovered, a signature mismatch against every oracle graph.

2. `accept-below-promise`: the acceptor accepts `Accept(b, v)` with `b < promised`. Expected: with promises for ballot 1 from a_0, a_1 and their accepts delayed, ballot 2 chooses v_2 through a_1, a_2, then a_0, a_1 accept the delayed ballot-1 v_1; disagreement; uncovered.

3. `silent-learner` (status): the winning proposer sends to one learner only. Expected: a learner blocked where the oracle's learners are both done; uncovered.

Reviewed (Opus, 2026-10-26): the rule, the bounds and all three faults check out; no schedule reports the conforming pair, the decided value's client reaching every learner directly or through adoption, and the oracle consuming only that client's proposal.

Why it is worth running. The shared protocol of the closest related work; the first family where the implementation's invisible nondeterminism (message order among five invisible threads) is large while the specification's is tiny (a choice among at most two values), the opposite of the load balancer.

2 B2 — Raft log replication, bounded, against an atomic log

Source. Verdi (PLDI 2015), whose headline is a verified Raft; Grove's replicated log abstractions; IronRSL's replicated state machine. The specification is the atomic log that state-machine replication refines.

What the pair checks. Three replicas run one term of Raft after a bounded election: r_0 and r_1 are candidates, r_2 votes for the first request it sees, so the leader is whichever candidate's request reaches r_2 first (there is no split at this size; two candidates with one voter decide by message order). Clients send commands through an invisible per-client front-end that fans the command out to every replica, so the client itself sends once; only the leader appends, replicates with `AppendEntries`, commits on

a majority of acks and replies to the client with the committed index; a visible reader asks, through its own front-end, for the committed log. The specification is one atomic log that may stop serving at any point.

Messages.

```
enum ToReplica { RequestVote(u8, usize), Vote(u8, bool), Cmd(usize /*client*/, i32),
                 AppendEntries(u8, usize /*leader*/, usize /*index*/, i32), Ack(u8, usize), Read }
enum ToClient  { Committed(usize) }
enum ToReader  { Log(Vec<i32>) }
```

Specification: the atomic log.

```
// visible: c0, c1, reader       invisible: log
let (tx_log, rx_log) = typed_chan::<ToReplica>();
let (tx_c, rx_c) = client_channels(2);
let (tx_r, rx_r) = typed_chan::<ToReader>();
named("log", move || {
    let mut log: Vec<i32> = Vec::new();
    for _ in 0..3 {                                  // two commands and one read, any order
        if crate::nondet() { return; }               // may stop here: the implementation may too (
    rule 1)
        match rx_log.recv_msg_block() {
            ToReplica::Cmd(c, v) => { log.push(v); tx_c[c].send_msg(ToClient::Committed(log.len()))
    ; }
            ToReplica::Read => tx_r.send_msg(ToReader::Log(log.clone())),
            _ => {}
        } }
});
for i in 0..2 { let tx = tx_log.clone(); let rx = rx_c[i];
    named(&format!("c{i}"), move || { tx.send_msg(ToReplica::Cmd(i, 100 + i as i32));
                                      let _r: ToClient = rx.recv_msg_block(); }); }
named("reader", move || { tx_log.send_msg(ToReplica::Read); let _l: ToReader = rx_r.recv_msg_block
    (); });
```

The stop in every iteration gives the specification a graph for every subset of clients left blocked, which the implementation reaches when a replica's non-blocking receive reads $\perp$ (rule 1). The read may observe zero, one or two commands.

Implementation: Raft, one term.

```
// visible: c0, c1, reader       invisible: f0, f1, fr (front-ends), r0, r1, r2
let (tx_rep, rx_rep) = replica_channels(3);
let (tx_f, rx_f) = frontend_channels(3);              // client i -> f_i, reader -> fr
let (tx_c, rx_c) = client_channels(2);
let (tx_r, rx_r) = typed_chan::<ToReader>();
for i in 0..3 { let rx = rx_f[i]; let tx_rep = tx_rep.clone();
    named(&format!("f{i}"), move || { let m: ToReplica = rx.recv_msg_block();
        for o in 0..3 { tx_rep[o].send_msg(m.clone()); } }); }          // fan-out (rule 3)
for me in 0..3 {
    let rx = rx_rep[me]; let tx_rep = tx_rep.clone(); let tx_c = tx_c.clone(); let tx_r = tx_r.
    clone();
    named(&format!("r{me}"), move || {
        let term: u8 = 1; let candidate = me < 2;
        let mut voted = false; let mut votes = if candidate { 1 } else { 0 };
        let mut leader: Option<usize> = None;
        let mut log: Vec<i32> = Vec::new(); let mut client_of: Vec<usize> = Vec::new();
        let mut acks: Vec<usize> = Vec::new(); let mut committed = 0usize;
        let mut seen: [bool; 2] = [false, false];       // one append per client (the fan-out)
        let mut waiting: Vec<(usize, i32)> = Vec::new(); // commands received before a leader is
    known
        let mut read_waiting = false;
        if candidate { for o in 0..3 { if o != me { tx_rep[o].send_msg(ToReplica::RequestVote(term,
     me)); } } }
        for _ in 0..BOUND {
            match rx.recv_msg() {
```

```
                Some(ToReplica::RequestVote(t, cand)) => {
                    let grant = t == term && !voted && !candidate; if grant { voted = true; }
                    tx_rep[cand].send_msg(ToReplica::Vote(t, grant)); }
                Some(ToReplica::Vote(t, true)) if t == term && candidate && leader.is_none() => {
                    votes += 1; if votes >= 2 { leader = Some(me); } }
                Some(ToReplica::Cmd(c, v)) => waiting.push((c, v)),
                Some(ToReplica::AppendEntries(t, l, idx, v)) if t == term => {
                    leader = Some(l);
                    if log.len() + 1 == idx { log.push(v); }
                    tx_rep[l].send_msg(ToReplica::Ack(t, idx)); }
                Some(ToReplica::Ack(t, idx)) if t == term && leader == Some(me) => {
                    acks[idx - 1] += 1;
                    while committed < log.len() && acks[committed] >= 2 {
                        committed += 1; tx_c[client_of[committed - 1]].send_msg(ToClient::Committed
    (committed)); } }
                Some(ToReplica::Read) => read_waiting = true,
                _ => {}
            }
            if leader == Some(me) {                          // the leader drains what it holds
                for (c, v) in waiting.drain(..) {
                    if seen[c] { continue; } seen[c] = true;
                    log.push(v); client_of.push(c); acks.push(1);
                    let idx = log.len();
                    for o in 0..3 { if o != me { tx_rep[o].send_msg(ToReplica::AppendEntries(term,
    me, idx, v)); } } }
                if read_waiting { read_waiting = false; tx_r.send_msg(ToReader::Log(log[..committed
    ].to_vec())); }
            } else if let Some(l) = leader {                 // a follower forwards what it holds
                for (c, v) in waiting.drain(..) { tx_rep[l].send_msg(ToReplica::Cmd(c, v)); }
                read_waiting = false;                         // the leader also received the Read
            }
        } });
}
for i in 0..2 { let tx = tx_f[i].clone(); let rx = rx_c[i];
    named(&format!("c{i}"), move || { tx.send_msg(ToReplica::Cmd(i, 100 + i as i32));
                                      let _r: ToClient = rx.recv_msg_block(); }); }
named("reader", move || { tx_f[2].send_msg(ToReplica::Read); let _l: ToReader = rx_r.recv_msg_block
    (); });
```

Commands and reads received before a leader is known are buffered locally, not re-sent, so the loop bound `BOUND` (about 10 at this size, derived by the tester) counts only real messages. The leader deduplicates the three copies of each command by client id; a follower's copy is forwarded and deduplicated the same way.

Differences. The specification has one thread and no election; its choices are the service order and where it stops. The implementation decides the leader by message order, replicates, and commits on acks. Reads at the leader return the committed prefix, which the specification's read is at the time of service.

Parameters. Replicas 3, clients 2, one command each, one reader, one term. Communication `p2p` (per-link FIFO, which Raft assumes). Growth: two commands per client.

Faults.

1. `commit-without-quorum`: the leader replies `Committed` on its own append. Not observable in one term without failures; a conforming control.

2. `out-of-order-commit`: the leader commits index 2 when its acks arrive before index 1's. Observable by status only: c_1 receives `Committed(2)` while c_0 blocks (index 1's acks read as $\perp$); the specification replying 2 to c_1 has already replied to c_0. Uncovered.

3. `follower-serves-read`: a follower answers `Read` from its own log. Observable twice: the reader gets `[100]` while c_0 is still blocked (uncommitted), or the losing candidate answers `[]` after both commits, where the specification's order from the read's send to c_0's receive is missing on the implementation side. Uncovered.

Why it is worth running. Verdi's protocol; the first family with an election and replication in one implementation. Reviewed: the first version of this pair was reported in every graph (clients sending three copies, replicas stopping on $\perp$ with no stop in the specification); this version follows rules 1 and 3.

3 B3 — Sharded key-value store with key migration against per-key stores

Source. IronFleet's IronKV (SOSP 2015), a sharded store whose shards move keys between hosts; Grove's reconfigurable store (SOSP 2023).

What the pair checks. Two shards hold the two keys; each client sends its requests through an invisible proxy that holds the client's routing table; an invisible controller moves key 0 from shard 0 to shard 1 during the run and tells the proxies. A request that reaches a shard that no longer owns the key is forwarded to the owner; a request that reaches the new owner before the transfer waits there. The specification is one store thread per key (rule 2: one thread for both keys would order every pair of operations, which two shards never do). The property is that the migration is invisible.

Messages and script. `Req::Put(c, k, v)`, `Req::Get(c, k)`, `Rep::Ack`, `Rep::Val(Option<i32>)`; the client script of A2 (`a2_op`: operation j of client i is a `Put` iff $i + j$ is even, on key $(\lfloor j/2 \rfloor + \lfloor i/2 \rfloor) \bmod 2$), $Q = 3$ so that key 0, the migrating key, carries both a `Put` and a `Get` from different clients.

Specification: one store per key.

```
// visible: c0, c1       invisible: store0, store1, x0, x1 (the proxies, unchanged on both sides)
let (tx_st, rx_st) = store_channels(2);          // one channel per key
let (tx_c, rx_c) = client_channels(2);
let (tx_x, rx_x) = proxy_channels(2);
for k in 0..2 { let rx = rx_st[k]; let tx_c = tx_c.clone();
    named(&format!("store{k}"), move || { let mut val: Option<i32> = None;
        loop { match rx.recv_msg_block() {                // blocking: ends blocked when the
    clients are done
            Req::Put(c, _, v) => { val = Some(v); tx_c[c].send_msg(Rep::Ack); }
            Req::Get(c, _) => tx_c[c].send_msg(Rep::Val(val)),
        } } }); }
for i in 0..2 { let rx = rx_x[i]; let tx_st = tx_st.clone();
    named(&format!("x{i}"), move || { for _ in 0..Q { let r: Req = rx.recv_msg_block();
        let k = match &r { Req::Put(_, k, _) | Req::Get(_, k) => *k as usize };
        tx_st[k].send_msg(r); } }); }
// clients: a2's script, each sending its Q requests to its proxy tx_x[i] and reading Q replies on
    rx_c[i]
```

Implementation: two shards, proxies with routing tables, a controller.

```
// visible: c0, c1       invisible: s0, s1, x0, x1, ctl
enum Msg { Req(Req), Move(u8, usize), Transfer(u8, Option<i32>) }
enum ToProxy { Req(Req), Owner(u8, usize) }
let (tx_s, rx_s) = shard_channels(2);              // Msg
let (tx_x, rx_x) = proxy_channels(2);              // ToProxy; the client sends ToProxy::Req
let (tx_c, rx_c) = client_channels(2);
named("ctl", move || { if crate::nondet() {        // migrate key 0 at some point, or never
    tx_s[0].send_msg(Msg::Move(0, 1)); for i in 0..2 { tx_x[i].send_msg(ToProxy::Owner(0, 1)); } }
    });
```

```
for me in 0..2 { let rx = rx_s[me]; let tx_s = tx_s.clone(); let tx_c = tx_c.clone();
    named(&format!("s{me}"), move || {
        let mut map: [Option<i32>; 2] = [None, None];
        let mut owns: [bool; 2] = [me == 0, me == 1];  // s0 owns key 0, s1 owns key 1
        let mut parked: Vec<Req> = Vec::new();         // requests for a key not yet transferred in
        loop { match rx.recv_msg_block() {
            Msg::Req(r) => { let k = key_of(&r) as usize;
                if owns[k] { serve(&mut map, &r, &tx_c); }
                else if me == 1 && k == 0 { parked.push(r); }      // the new owner, before
    Transfer
                else { tx_s[1 - me].send_msg(Msg::Req(r)); } }     // forward to the owner
            Msg::Move(k, to) => { owns[k as usize] = false;
                tx_s[to].send_msg(Msg::Transfer(k, map[k as usize])); }
            Msg::Transfer(k, v) => { map[k as usize] = v; owns[k as usize] = true;
                for r in parked.drain(..) { serve(&mut map, &r, &tx_c); } }
        } } }); }
for i in 0..2 { let rx = rx_x[i]; let tx_s = tx_s.clone();
    named(&format!("x{i}"), move || { let mut owner: [usize; 2] = [0, 1];
        let mut served = 0;
        while served < Q { match rx.recv_msg_block() {
            ToProxy::Owner(k, to) => owner[k as usize] = to,
            ToProxy::Req(r) => { served += 1; tx_s[owner[key_of(&r) as usize]].send_msg(Msg::Req(r)
    ); }
        } } }); }
// clients: a2's script through tx_x[i], replies on rx_c[i]; serve() applies a Put or Get and
    replies
```

The proxy's `while` is bounded by the number of requests, with at most one `Owner` message interleaved, so it unrolls. Under per-link FIFO the `Transfer` on the link $s_0 \to s_1$ precedes every request s_0 forwards after `Move`, so a forwarded request never bounces; a request that reaches s_1 through a proxy that already learnt the new owner may still precede the `Transfer` and is parked.

Differences. The specification has one store per key and no migration; the implementation has two shards, a moving key, stale routing tables, forwarded and parked requests. Both order only same-key operations, through the owner or through the transfer.

Parameters. Shards 2, keys 2, clients 2, $Q = 3$, one migration. Communication `p2p`. Growth: three clients, or the migration of both keys.

Faults (all on key 0, the migrating one, with $Q = 3$ so that c_0 puts and c_1 gets on it).

1. `stale-after-move`: s_0 keeps serving `Get` for key 0 from its copy after `Move`. Schedule: c_0's `Put` is served at s_1 after the transfer, then c_1's `Get` reaches s_0 through a stale proxy and is answered from the old copy: `None` or an older value. Uncovered (value).

2. `lost-transfer`: s_1 takes ownership on `Move` without waiting for `Transfer` and serves `None`. Uncovered (value) after a `Put` at s_0.

3. `double-apply`: a forwarded `Put` is applied and acked by both shards. The client receives two acks; the second is read as the reply to its next request, a value mismatch there. Uncovered.

Why it is worth running. IronKV's protocol, and a key-value pair where the implementation's extra invisible structure changes during the run. Reviewed: the first version was reported in every graph (a visible client receiving routing updates; one store thread for both keys; bounded non-blocking shards) and its faults were unreachable (key 1 migrated, the script never put on it); this version follows rules 1 to 4.

4 B4 — Causally consistent replicated key-value store against per-replica causal servers

Source. Chapar (POPL 2016), certified causally consistent key-value stores; the Aneris follow-up on a causally consistent store (POPL 2022).

What the pair checks. Two replicas, each with one client; a client puts and gets at its own replica; replicas exchange updates over a reordering network and apply a remote update only after its dependencies. The specification has one server per replica too (rule 2: a single oracle would order the two clients' operations), each serving its own client in program order and receiving the other's updates, but applying a remote update at a moment it chooses, subject to causality. So the specification's extra freedom is exactly the delay a network may add.

Script. Client c_0: `Put(x, 1)`, `Put(y, 2)`, `Get(y)`. Client c_1: `Get(y)`, `Get(x)`, `Put(x, 3)`. Keys $x = 0$, $y = 1$. A read of $y = 2$ at r_1 followed by a read of $x =$ `None` is the causal violation the faults produce.

Messages.

```
enum Op  { Put(u8, i32), Get(u8) }
enum Rep { Ack, Val(Option<i32>) }
struct Update { key: u8, val: i32, origin: usize, deps: [usize; 2] }
enum Msg { Op(Op), Upd(Update) }
```

Specification: two causal servers.

```
// visible: c0, c1       invisible: v0, v1
let (tx_v, rx_v) = server_channels(2);     // Msg; the client's ops and the other server's updates
    (NoOrder)
let (tx_c, rx_c) = client_channels(2);
for me in 0..2 { let rx = rx_v[me]; let tx_v = tx_v.clone(); let tx_c = tx_c.clone();
    named(&format!("v{me}"), move || {
        let mut view: [Option<i32>; 2] = [None, None]; let mut clock: [usize; 2] = [0, 0];
        let mut pending: Vec<Update> = Vec::new();
        loop {
            // before serving, apply any ready pending update, or not, freely (the network's delay)
            let mut i = 0;
            while i < pending.len() { let u = pending[i].clone();
                let ready = u.deps[u.origin] == clock[u.origin] + 1 && u.deps[1 - u.origin] <=
    clock[1 - u.origin];
                if ready && crate::nondet() { view[u.key as usize] = Some(u.val); clock[u.origin]
    += 1; pending.remove(i); }
                else { i += 1; } }
            match rx.recv_msg_block() {
                Msg::Op(Op::Put(k, v)) => { clock[me] += 1; view[k as usize] = Some(v);
                    tx_v[1 - me].send_msg(Msg::Upd(Update { key: k, val: v, origin: me, deps: clock
     }));
                    tx_c[me].send_msg(Rep::Ack); }
                Msg::Op(Op::Get(k)) => tx_c[me].send_msg(Rep::Val(view[k as usize])),
                Msg::Upd(u) => pending.push(u),
            } } }); }
for i in 0..2 { /* the clients, each sending its three ops to tx_v[i] as Msg::Op and reading three
    replies */ }
```

Implementation: two replicas with dependency tracking.

```
// visible: c0, c1       invisible: r0, r1
// identical to the servers above with one difference: a pending update whose dependencies are
// met is applied at once (no nondet), the network's reordering being the only source of delay.
```

Differences. The two sides are structurally the same two threads; the specification may delay a ready update, the implementation applies it at once. Since the update channel reorders, the implementation's graphs are the specification's graphs in which the delay is realised by the network, and the specification has more.

Parameters. Replicas 2, clients 2, three operations each, keys 2. The update channels `NoOrder`; the client channels FIFO. Growth: three replicas with three-entry clocks.

Faults.

1. `apply-eagerly`: the replica applies a buffered update without checking its dependencies. Schedule: c_0 puts x then y; the update for y overtakes the one for x on the reordering channel; r_1 applies y at once; c_1 reads $y = 2$ then $x =$ `None`. The specification can never return that pair. Uncovered (value).

2. `lost-update-with-clock`: the replica advances its clock for a ready update but does not write the value. Schedule: as above with the updates in order; c_1 reads $y =$ `None` after r_1 has advanced past it, and a later dependent update applies on top of the missing one, so the final read differs from every specification graph. Uncovered (value). The variant that drops the update without advancing the clock is not observable, since the specification may delay forever; it is a conforming control.

The fault `wrong-origin-clock` of the first version is dropped: with two replicas it only delays updates, which the specification allows.

Why it is worth running. Chapar's property as a specification program; the pair where the specification is the implementation plus a choice, which makes the index's containment test do the work. Reviewed: the first version had one oracle (reported in the common schedule) and a script that never read a written key; this version follows rules 2 and 4. Estimate revised from 2 to 3 lead days.

5 B5 — A Memcached-style server with connections against one map

Source. Bílý et al. (OOPSLA 2025, §5.1, Memcached v1–v3 with Get, Set, Delete over connections, one thread per connection, a store behind a lock). Extends our A2.

What the pair checks. Each client holds one connection and issues a sequence of `Get`, `Set` and `Delete` on two keys. The implementation has one worker per connection; the workers share no memory and reach the store through a storage thread by message passing, which is where the server's internal concurrency lives. The specification is one map.

Messages.

```
enum Cmd { Get(u8), Set(u8, i32), Delete(u8) }
enum Res { Val(Option<i32>), Stored, Deleted, NotFound }
```

Specification: one map (as `a2_spec`, with the `Delete` arm: `Deleted` if the key was present, `NotFound` otherwise; both clients send `(i, Cmd)` to one channel, served in arrival order, the reply to client i). Visible: the clients. Invisible: `store`. One store thread is right here (rule 2): the implementation's single storage thread orders every pair of commands too.

Implementation: a worker per connection and a storage thread.

```
// visible: c0, c1       invisible: w0, w1, storage
let (tx_w, rx_w) = worker_channels(2);      // client i -> worker i (its connection)
let (tx_st, rx_st) = typed_chan::<(usize, Cmd)>();   // workers -> storage
let (tx_wr, rx_wr) = worker_reply_channels(2);        // storage -> worker i
let (tx_c, rx_c) = client_channels(2);
named("storage", move || {
    let mut map: [Option<i32>; 2] = [None, None];
    for _ in 0..2 * Q {
        let (w, cmd) = rx_st.recv_msg_block();
        let res = match cmd {
            Cmd::Get(k) => Res::Val(map[k as usize]),
            Cmd::Set(k, v) => { map[k as usize] = Some(v); Res::Stored }
            Cmd::Delete(k) => if map[k as usize].take().is_some() { Res::Deleted } else { Res::
    NotFound },
        };
        tx_wr[w].send_msg(res);
    }
});
for w in 0..2 {
    let rx = rx_w[w]; let tx_st = tx_st.clone(); let rx_wr = rx_wr[w]; let tx_c = tx_c.clone();
    named(&format!("w{w}"), move || { for _ in 0..Q {
        let (c, cmd): (usize, Cmd) = rx.recv_msg_block();   // the same (i, Cmd) the spec receives
        tx_st.send_msg((c, cmd));
        let res: Res = rx_wr.recv_msg_block();
        tx_c[w].send_msg(res);
    } });
}
for i in 0..2 { let tx = tx_w[i].clone(); let rx = rx_c[i];
    named(&format!("c{i}"), move || {                       // both connections on key 0: Set, Get,
     Delete
        for cmd in [Cmd::Set(0, 10 + i as i32), Cmd::Get(0), Cmd::Delete(0)] {
            tx.send_msg((i, cmd)); let _r: Res = rx.recv_msg_block(); } }); }
```

Differences. The specification serves commands in one arrival order; the implementation interleaves two workers in front of one storage thread, so the service order is the storage thread's arrival order, which the workers' relays reshuffle. With a store behind a single thread the pair conforms; the realistic faults are in the workers.

Parameters. Clients 2, $Q = 3$ (`Set`, `Get`, `Delete`, both on key 0, so that the connections interact). Communication `p2p` (TCP connections). Growth: three clients, or a second key with a fourth command.

Faults.

1. `cached-get`: a worker answers a `Get` on key k from the value its own connection last `Set` on k, without asking the storage. Schedule: c_0 sets 10, c_1 sets 11, c_0 gets and is answered 10 from the worker's cache while the storage holds 11; the specification serving c_0's `Get` after c_1's `Set` returns 11, and serving it before has the order c_0's get before c_1's set, absent from the implementation graph. Uncovered.

2. `delete-ack-early`: the worker replies `Deleted` without asking the storage. Schedule: c_0 sets, c_0 deletes (`Deleted`), c_1 deletes and is also answered `Deleted` by its worker, where the storage would say `NotFound`; no specification order gives two `Deleted` after one `Set`. Uncovered.

The fault `reorder-connection` of the first version is dropped: the clients are closed-loop, so a worker never holds two commands.

Why it is worth running. The case study of the closest related work, written in our language; against their 118 to 181 lines of Rust and 286 to 377 lines of proof annotations, our pair is two programs of this size and no annotations.

6 B6 — Ring leader election against an oracle naming the maximum

Source. Igloo (OOPSLA 2020) verifies a leader election protocol among its three case studies; the ring election of Chang and Roberts is the standard one. We have a leader election already (A5, within two-phase commit); this one is the classic ring and stands alone.

What the pair checks. n nodes in a ring, each with a unique identifier; each node sends its identifier to its successor; a node forwards an identifier larger than its own, drops a smaller one, and the node that receives its own identifier is the leader and sends `Leader(id)` around the ring once. Every node has a visible observer that receives the announcement. The specification is an oracle that sends `Leader(max id)` to every observer.

Specification.

```
// visible: o0..o(n-1)       invisible: oracle
let (tx_o, rx_o) = observer_channels(n);
named("oracle", move || { let m = IDS.iter().max().unwrap(); for tx in &tx_o { tx.send_msg(Leader(*
    m)); } });
for (i, rx) in rx_o.into_iter().enumerate() { named(&format!("o{i}"), move || { let _l: Leader = rx
    .recv_msg_block(); }); }
```

Implementation: Chang–Roberts.

```
// visible: o0..o(n-1)       invisible: n0..n(n-1)
let (tx_n, rx_n) = node_channels(n);
let (tx_o, rx_o) = observer_channels(n);
for me in 0..n {
    let rx = rx_n[me]; let succ = tx_n[(me + 1) % n].clone(); let tx_obs = tx_o[me].clone();
    named(&format!("n{me}"), move || {
        let my = IDS[me];
        succ.send_msg(Ring::Elect(my));
        for _ in 0..n {                                  // at most n election messages pass a
    node
            match rx.recv_msg_block() {
                Ring::Elect(id) if id > my => succ.send_msg(Ring::Elect(id)),
                Ring::Elect(id) if id == my => { succ.send_msg(Ring::Leader(my)); tx_obs.send_msg(
    Leader(my)); return; }
                Ring::Elect(_) => {}                      // smaller: drop
                Ring::Leader(id) => { tx_obs.send_msg(Leader(id)); if id != my { succ.send_msg(Ring
    ::Leader(id)); } return; }
            } }
    });
}
for (i, rx) in rx_o.into_iter().enumerate() { named(&format!("o{i}"), move || { let _l: Leader = rx
    .recv_msg_block(); }); }
```

With `IDS` a fixed permutation, e.g. $[2, 0, 3, 1]$. The loop bound n covers the longest path an identifier travels; the leader's own `Leader` message returns to it and the `return` ends the thread.

Differences. The specification knows the maximum; the implementation discovers it by forwarding, with the message interleavings of n nodes. The observable is one value per observer and the statuses.

Parameters. $n \in \{3, 4, 5\}$. Communication `p2p` (a ring link is FIFO). Growth: $n = 6$, and the variant with two identifiers equal (a tie the protocol must not have).

Faults.

1. `forward-smaller`: the node holding the maximum forwards a smaller identifier instead of dropping it. Expected: the second-largest identifier returns to its owner, which announces itself; observers see a wrong leader; uncovered (value). Placed at any other node the fault is a conforming control.

2. `drop-announcement`: a node other than the leader's predecessor does not forward `Leader`. Expected: the observers beyond it block; uncovered (status). At the leader's predecessor the announcement has already gone round, a conforming control.

3. `announce-twice`: the leader forwards its own `Leader` message again on return. Expected: an observer receives two announcements where the oracle sends one; the second is pending and unobserved, so this is a conforming control at this size.

Why it is worth running. Igloo's protocol; the cheapest pair in this file (one afternoon), and one where the specification has a single graph. Reviewed: correct under per-link FIFO, the bound n tight and sufficient.

7 Summary and order of implementation

pair	protocol	source	smallest size
B1	single-decree Paxos	IronFleet, Trillium, MoDist, OOPSLA'25	2 proposers, 3 acceptors, 2 cli
B2	Raft replication, one term	Verdi	3 replicas, 2 clients, 1 reader
B3	sharded store with migration	IronKV, Grove	2 shards, 2 clients, $Q = 3$
B4	causally consistent store	Chapar	2 replicas, 2 clients, 3 ops eac
B5	Memcached-style server	OOPSLA'25	2 clients, $Q = 3$
B6	ring leader election	Igloo	$n = 4$

Order by value for the paper: B1, B6, B5, B3, B2, B4. B1 is the shared protocol of the closest related work and reviewed sound; B6 is cheap and reviewed sound; B5 is their case study in our language; B3 and B2 are the realistic protocols of the deductive line, each three to four days; B4 last, since only two of its faults are observable and its specification is nearly its implementation. Each pair goes to the lead as one item: the tester derives the expected verdicts and uncovered sets at the smallest size by hand before any run, the conforming pair and every fault are run under both engines at the smallest size and one size up, and the digest records, as for A1–A5, the graph counts of both sides, the wall times, the peaks and the first-report times. Nothing here runs before the final campaign plan names it.